1 Introduction

The performance of Artificial Intelligence (AI)-driven network slicing algorithms is fundamentally dependent on the availability of representative datasets that accurately capture the characteristics of heterogeneous network traffic. Existing publicly available datasets primarily focus on intrusion detection, traffic classification, anomaly detection, or quality-of-service prediction. While these datasets have significantly contributed to networking research, they do not adequately represent the service-oriented communication patterns required for intelligent resource allocation in next-generation mobile networks.

The emergence of Fifth Generation (5G) and Beyond Fifth Generation (B5G) networks introduces fundamentally different communication paradigms compared to conventional IP networks. Modern networks simultaneously support latency-critical healthcare applications, bandwidth-intensive educational services, delay-tolerant agricultural Internet of Things (IoT) deployments, and conventional Internet traffic. Consequently, intelligent network slicing algorithms require datasets that describe not only packet-level characteristics but also the contextual information influencing network behaviour.

Most publicly available traffic datasets exhibit one or more of the following limitations:

- They focus primarily on cybersecurity applications rather than service-aware resource allocation.
- They contain application labels without explicitly modelling network slice requirements.
- They lack contextual information describing radio conditions, user mobility, network congestion, or infrastructure utilisation.
- They provide static packet traces collected under limited operating conditions instead of representing multiple network scenarios.
- They are generally collected from enterprise or campus environments rather than rural communication infrastructures.

These limitations significantly reduce their applicability for training AI models intended for intelligent network slicing in rural deployments. In particular, reinforcement learning agents require environment-aware state representations, while supervised learning models require service-specific traffic labels accompanied by realistic Quality of Service (QoS) requirements. Existing datasets rarely satisfy both requirements simultaneously.

To address these shortcomings, this work proposes the **Rural Service-aware Network Slicing Dataset (RSNSD)**, a synthetic yet statistically realistic dataset specifically designed for AI-assisted network slicing research. Rather than replaying existing packet traces, the proposed framework generates traffic using parameterised stochastic models representing realistic application behaviour. This enables the generation of diverse network flows while preserving reproducibility and allowing controlled experimentation under multiple network scenarios.

Unlike traditional datasets that only capture packet headers or flow statistics, RSNSD incorporates three complementary information layers:

1. Traffic Characteristics describing packet- and flow-level behaviour.

2. Network Context describing radio conditions, infrastructure utilisation, and user mobility.

3. Service Intelligence describing application classes, network slice assignments, Quality of Service requirements, and scheduling labels.

The proposed dataset is specifically designed to support both supervised and reinforcement learning paradigms. Supervised learning models utilise traffic characteristics and contextual information to classify service categories, while reinforcement learning algorithms employ the contextual state information to learn adaptive slice allocation policies under varying network conditions.

Furthermore, the dataset architecture has been designed using a modular generation pipeline consisting of independent traffic models, flow generation modules, context generation modules, Quality of Service modelling, schema validation, and dataset export mechanisms. This modular architecture enables future extensions without requiring modifications to the overall framework.

The remainder of this chapter presents the complete design methodology adopted for RSNSD, including the dataset architecture, feature engineering process, metadata specification, traffic modelling strategy, contextual scenario generation, schema design, validation procedures, and extensibility considerations.

2 Dataset Design Objectives

The primary objective of the proposed Rural Service-aware Network Slicing Dataset (RSNSD) is to provide a comprehensive and extensible dataset for Artificial Intelligence (AI)-driven resource management in rural Fifth Generation (5G) communication networks. Unlike conventional network traffic datasets that primarily focus on packet inspection, intrusion detection, or generic traffic classification, RSNSD has been specifically designed to facilitate intelligent network slicing, service-aware scheduling, Quality of Service (QoS) optimization, and reinforcement learning based resource allocation.

The design philosophy of RSNSD follows the principle that a dataset should accurately represent both the characteristics of generated traffic and the operational environment in which that traffic exists. Consequently, the proposed dataset captures multiple dimensions of network behaviour, including packet-level information, flow-level statistics, temporal behaviour, Quality of Service characteristics, radio conditions, infrastructure utilisation, user mobility, and service-specific contextual information.

To achieve these goals, the following primary design objectives were established during the development of the dataset.

2.1 Primary Objectives

1. **Develop a Service-aware Dataset**

   The foremost objective is to create a dataset capable of distinguishing network services rather than merely identifying transport protocols or applications. Each generated flow is associated with a well-defined service class, enabling machine learning algorithms to learn service-specific communication patterns that are directly applicable to intelligent network slicing.

2. **Support AI-driven Network Slicing**

   The dataset is designed specifically for supervised learning and reinforcement learning algorithms operating within Software Defined Networking (SDN) and 5G network slicing environments. All features have therefore been selected to maximize their usefulness for intelligent slice selection and adaptive resource allocation.

3. **Model Rural Communication Networks**

   Unlike enterprise traffic datasets, RSNSD models communication scenarios commonly observed in rural deployments, including agricultural IoT systems, telemedicine consultations, online education platforms, emergency healthcare communication, and general Internet access. This allows AI models trained using the dataset to better generalize towards rural smart infrastructure deployments.

4. **Capture Multi-layer Network Behaviour**

   Rather than restricting the dataset to packet headers or flow statistics, RSNSD captures information across multiple protocol layers and contextual dimensions. The resulting feature space combines Layer-2, Layer-3, Layer-4, flow-level, temporal, statistical, QoS, radio, mobility, and network context features into a unified representation.

5. **Ensure Research Reproducibility**

   The dataset generation pipeline has been designed using deterministic stochastic models with configurable random seeds. This enables researchers to reproduce identical datasets under identical experimental conditions while simultaneously allowing controlled variation through configurable network scenarios.

2.2 Secondary Objectives

In addition to the primary goals, several secondary objectives guided the architectural decisions throughout the dataset development process.

1. Modular software architecture allowing independent development of traffic generation, context modelling, feature extraction, validation, and export modules.

2. Extensible application profiles enabling future integration of additional services without modification of the existing dataset generation framework.

3. Automatic schema validation to ensure feature consistency across multiple dataset versions.

4. Compatibility with Open5GS, Mininet, Open vSwitch, and software-defined networking testbeds.

5. Straightforward integration with machine learning frameworks such as Scikit-Learn, XGBoost, TensorFlow, and PyTorch.

6. Support for future real-world calibration using traffic collected from the proposed institutional 5G testbed.

2.3 Design Principles

The architectural design of RSNSD was governed by several engineering principles intended to improve maintainability, reproducibility, and scientific validity.

- **Modularity:** Every functional component of the dataset generation framework is implemented as an independent software module with clearly defined interfaces.
- **Separation of Concerns:** Traffic generation, context modelling, Quality of Service computation, feature extraction, schema management, and dataset export are implemented independently to simplify future extensions.
- **Statistical Realism:** Network characteristics are generated using parameterized probability distributions rather than deterministic values, enabling realistic variability across generated flows.
- **Scenario Awareness:** Instead of generating isolated traffic samples, the dataset models complete network operating conditions such as normal operation, school hours, harvest season, and disaster response scenarios.
- **Schema-first Development:** The dataset schema was designed before implementing the traffic generation framework, ensuring that every generated feature corresponds to a predefined and validated metadata specification.
- **Research-oriented Extensibility:** Every module has been designed to facilitate future integration of additional service categories, communication technologies, protocol layers, and machine learning workflows.

2.4 Success Criteria

The success of the proposed dataset is evaluated using both engineering and research-oriented metrics.

From an engineering perspective, the dataset generation framework must be capable of producing internally consistent network flows whose features satisfy the predefined schema while remaining fully reproducible.

From a machine learning perspective, the dataset should enable accurate service classification, efficient network slice prediction, and reinforcement learning based scheduling using a unified feature representation.

Finally, from a research perspective, the proposed dataset should provide sufficient flexibility to evaluate intelligent network slicing algorithms under multiple rural communication scenarios while remaining extensible towards future Beyond Fifth Generation (B5G) and Sixth Generation (6G) communication systems.

3 Overall Dataset Architecture

The Rural Service-aware Network Slicing Dataset (RSNSD) has been designed as a modular data generation framework rather than a conventional packet capture repository. Instead of relying on previously collected traffic traces, the proposed framework synthesizes statistically realistic network flows by modelling application behaviour, network context,

Quality of Service (QoS) dynamics, and service-aware labels through a sequence of independent software modules.

The modular architecture offers three major advantages. First, each functional component can be independently validated, tested, and extended without affecting the remaining pipeline. Second, the separation of concerns improves reproducibility by isolating traffic generation from contextual modelling and dataset export. Finally, the architecture enables future integration with real-world traffic sources collected from institutional 5G testbeds while preserving compatibility with the existing schema.

Figure 1 illustrates the overall architecture adopted for the proposed dataset generation framework.

Figure 1: Overall architecture of the RSNSD generation framework.

The dataset generation process begins with the definition of service-specific application profiles. Each application profile captures the statistical characteristics of an individual communication service, including transport protocol, packet size distribution, session duration, bitrate, Quality of Service requirements, mobility characteristics, and target network slice. Rather than assigning deterministic values, each characteristic is represented using parameterized probability distributions to emulate realistic traffic variability.

The application profiles are registered within a centralized profile registry, which serves as the single source of truth for all supported communication services. This registry allows new services to be incorporated into the framework without modifying the remaining dataset generation pipeline.

During dataset generation, the Flow Generator samples the statistical distributions associated with an application profile to produce synthetic network flows. These flows represent the intrinsic behaviour of an application independent of the surrounding network conditions. Generated flow characteristics include packet sizes, inter-arrival times, session durations, bitrates, packet counts, and byte counts.

Following flow generation, the Context Generator augments each flow with information describing the operational state of the network. Context generation is scenario-driven rather than purely stochastic. Instead of generating arbitrary values, the generator produces network context consistent with predefined rural operating conditions such as normal operation, school hours, harvest season, and disaster response. Each scenario influences network load, active user population, radio quality indicators, slice utilization, and mobility characteristics, thereby creating internally consistent operating environments.

The generated flow and contextual information are subsequently processed by the Quality of Service Generator. Rather than assigning QoS metrics independently, this module derives latency, jitter, throughput, and packet loss from both application requirements and current network conditions. Consequently, the observed Quality of Service becomes a function of traffic characteristics, network congestion, and radio environment, thereby preserving causal relationships within the generated dataset.

Once traffic characteristics, contextual information, and Quality of Service metrics have been generated, the Label Assignment module determines the supervisory information associated with each flow. These labels include service class, network slice type, service priority, scheduling decisions, and reinforcement learning reward information. The separation of input features from supervisory labels enables the resulting dataset to support both supervised learning algorithms and reinforcement learning based resource allocation.

Before export, every generated feature is validated against the predefined dataset schema. The schema validation module ensures that feature names, data types, normalization strategies, metadata, and machine learning roles remain consistent throughout the generation process. This schema-first design philosophy prevents inconsistencies between generated data and dataset documentation while ensuring long-term maintainability.

Finally, validated flow records are exported into structured dataset formats suitable for downstream machine learning pipelines. Although the initial implementation targets comma-separated value (CSV) files, the modular exporter architecture permits future support for additional storage formats such as Apache Parquet, JavaScript Object Notation (JSON), or packet capture (PCAP) representations.

The proposed architecture deliberately separates traffic generation, contextual modelling, Quality of Service computation, feature validation, and dataset export into independent modules. This separation not only improves software maintainability but also facilitates controlled experimentation by allowing individual components to be modified or replaced without affecting the remainder of the framework. Such modularity is particularly valuable for future extensions involving real network traces, alternative radio access technologies, or Beyond Fifth Generation (B5G) communication systems.

4 Dataset Generation Pipeline

The proposed Rural Service-aware Network Slicing Dataset (RSNSD) is generated through a multi-stage data generation pipeline that transforms high-level application definitions into machine learning ready flow records. Unlike conventional datasets collected through passive network monitoring, RSNSD follows a synthetic generation methodology in which every generated feature is derived from statistically modelled application behaviour and dynamically generated network conditions.

The pipeline has been designed as a sequence of loosely coupled modules, where each module performs a single well-defined task before passing the generated information to the subsequent stage. This modular execution model simplifies validation, facilitates future extensions, and enables individual components to be independently calibrated using real network measurements.

Figure 2 illustrates the execution flow adopted by the proposed framework.

Figure 2: Runtime execution pipeline of the RSNSD generation framework.

The execution pipeline begins by selecting an application profile from the centralized profile registry. Each application profile encapsulates the statistical communication characteristics of a particular service, including transport protocol, packet size distribution, inter-arrival behaviour, bitrate characteristics, session duration, mobility profile, Quality of Service requirements, and target network slice. The application profile therefore represents the behavioural template from which synthetic traffic instances are generated.

Once an application profile has been selected, the framework samples each of its statistical distributions using the sampling engine. Unlike deterministic generators that repeatedly produce identical traffic patterns, the proposed framework employs parameterized probability distributions to generate statistically diverse yet reproducible traffic instances. Consequently, every generated network flow exhibits natural variations while remaining consistent with the expected behaviour of its corresponding application.

The sampled values are then passed to the Flow Generator, which constructs a synthetic network flow by combining packet-level and flow-level characteristics into a unified flow representation. This stage produces the intrinsic communication characteristics of the application independent of the surrounding network environment.

Following flow generation, the Context Generator augments the generated flow with environmental information representing the operational state of the communication network. Context generation is performed using predefined network scenarios rather than independent random variables. Each scenario models realistic rural communication conditions such as normal operation, school hours, agricultural harvesting periods, or emergency disaster response. These scenarios influence multiple contextual parameters simultaneously, thereby preserving correlations between network utilisation, radio conditions, user density, and slice resource consumption.

The generated contextual information includes network load, serving cell utilisation, active user population, mobility state, radio signal quality, Channel Quality Indicator (CQI), Signal-to-Interference-plus-Noise Ratio (SINR), Reference Signal Received Power (RSRP), Reference Signal Received Quality (RSRQ), slice utilisation, radio resource allocation, and handover status. Since these parameters originate from a common network scenario, the resulting contextual information remains internally consistent throughout the generated dataset.

The subsequent Quality of Service (QoS) Generation stage computes the observed communication performance experienced by each generated flow. Instead of assigning latency, jitter, throughput, and packet loss independently, these metrics are modelled as functions of both the application requirements and the generated network context. This design introduces causal relationships between traffic characteristics and observed network performance, thereby improving the realism of the generated dataset.

Once the traffic characteristics and Quality of Service metrics have been generated, the Label Assignment module associates each flow with the corresponding supervisory information required by downstream machine learning models. The generated labels include service class, application identifier, network slice type, service priority, scheduling decisions, and reinforcement learning reward information. The separation of predictive features from supervisory labels enables the resulting dataset to support both supervised learning based traffic classification and reinforcement learning based network resource allocation.

Prior to export, every generated flow is validated against the predefined dataset schema. The schema validation stage verifies feature names, data types, normalization strategies, feature groups, metadata specifications, and machine learning roles. This validation process ensures that every exported dataset remains fully consistent with the documented schema while preventing accidental introduction of invalid or duplicate features during future extensions.

Finally, validated flow records are exported into structured datasets suitable for downstream machine learning pipelines. Although the current implementation targets comma-separated value (CSV) files due to their widespread compatibility with machine learning frameworks, the modular exporter architecture allows future support for JavaScript Object Notation (JSON), Apache Parquet, or Packet Capture (PCAP) formats without requiring modifications to the preceding generation pipeline.

The proposed execution pipeline intentionally separates traffic modelling, contextual modelling, Quality of Service computation, schema validation, and dataset export into independent stages. This separation enables individual modules to evolve independently

while preserving the overall integrity of the framework. Furthermore, the modular design simplifies experimentation by allowing researchers to replace or calibrate individual components using measurements collected from operational 5G testbeds without modifying the remainder of the generation process.

5 Schema Design

The schema represents the formal specification governing the structure of the Rural Service-aware Network Slicing Dataset (RSNSD). Rather than treating the dataset as an unstructured collection of features, the proposed framework adopts a schema-first design methodology in which every generated attribute is explicitly defined before the implementation of the traffic generation pipeline.

The schema serves as the contractual interface between all modules of the dataset generation framework. Every component, including traffic generation, context modelling, Quality of Service computation, feature extraction, validation, machine learning, and dataset export, interacts exclusively through the predefined schema. Consequently, the schema ensures structural consistency throughout the lifetime of the project while significantly improving maintainability and reproducibility.

Unlike conventional datasets where feature definitions are implicitly embedded within source code or external documentation, RSNSD maintains a centralized metadata specification describing every dataset attribute. This metadata-driven approach allows the framework to automatically validate generated records, generate documentation, and maintain compatibility between multiple versions of the dataset.

Figure 3 illustrates the schema-centric architecture adopted by the proposed framework.

Figure 3: Metadata-centric schema architecture adopted by RSNSD.

The schema consists of a collection of feature definitions, where every feature is represented as an independent metadata object describing its semantic meaning, datatype, source, normalization strategy, machine learning role, and associated protocol layer. Instead of directly manipulating raw column names, all dataset modules operate using these metadata objects, thereby reducing inconsistencies and simplifying future extensions.

Each feature definition is represented using a dedicated `Feature` object containing the complete metadata specification required throughout the dataset generation process. The principal attributes associated with every feature include:

- Feature name
- Human-readable description
- Feature category
- Data type
- Measurement unit
- Valid numerical bounds
- Nullable property

- Normalization strategy
- Feature source
- Machine learning role
- Derived feature indicator

Collectively, these attributes provide a complete semantic description of every dataset column while remaining independent of the generated data itself.

5.1 Schema-first Design Philosophy

A fundamental design principle adopted throughout the development of RSNSD is the schema-first approach. Instead of generating traffic and subsequently deciding which attributes should be preserved, the complete feature space was designed prior to implementing any traffic generation components.

This design philosophy offers several advantages.

1. Every generated feature corresponds to a predefined metadata specification.
2. Dataset generators cannot introduce undocumented attributes.
3. Validation becomes deterministic since every generated value is verified against the predefined schema.
4. Dataset documentation can be generated automatically from the metadata definitions.
5. Future extensions remain backward compatible through schema versioning.

The schema therefore functions as the central contract governing every stage of dataset generation.

5.2 Metadata-driven Dataset Representation

Instead of storing only numerical values, RSNSD explicitly separates feature metadata from feature values. Each generated flow therefore consists of two complementary components:

1. Metadata describing the semantics of every feature.
2. Observed values generated during traffic synthesis.

This separation significantly improves maintainability since modifications to feature descriptions, normalization strategies, measurement units, or machine learning roles do not require changes to the traffic generation pipeline.

Furthermore, the metadata representation enables automatic generation of documentation, validation rules, feature catalogues, and dataset schemas without manual intervention.

5.3 Modular Schema Organization

To improve maintainability, the complete feature space has been partitioned into independent feature modules based on their semantic responsibilities rather than their implementation order.

Each module represents a logically coherent subset of the complete dataset and can therefore evolve independently while remaining compatible with the remainder of the framework.

The major schema modules include:

- Layer-2 Features
- Layer-3 Features
- Layer-4 Features
- Flow Features
- Temporal Features
- Statistical Features
- Quality of Service Features
- Context Features
- Label Features

This modular organization enables individual feature groups to be maintained independently while simplifying future integration of additional protocol layers or contextual information.

5.4 Schema Validation

The schema incorporates an automated validation framework executed prior to dataset generation. Instead of relying on manual inspection, every feature definition undergoes consistency verification before the schema is exported.

The validation process currently verifies:

- Duplicate feature names.
- Invalid numerical ranges.
- Missing feature descriptions.
- Missing machine learning roles.
- Metadata consistency across feature definitions.

Performing validation prior to dataset generation prevents invalid schema definitions from propagating into downstream machine learning experiments.

5.5 Automatic Schema Generation

Rather than manually maintaining multiple metadata files, RSNSD employs a centralized schema generator responsible for producing all schema-related artifacts directly from the feature registry.

The generator automatically exports:

- Complete schema definition (`schema.json`)
- Human-readable feature catalogue (`feature_catalog.md`)
- Machine learning label specification (`labels.json`)

This automated generation process guarantees consistency between implementation and documentation while eliminating duplicate maintenance effort.

5.6 Advantages of the Proposed Schema

Compared to conventional datasets where metadata is scattered across implementation files or external documentation, the proposed schema offers several significant advantages.

- Centralized metadata management.
- Strong separation between metadata and generated values.
- Automatic validation before dataset generation.
- Automatic documentation generation.
- Extensible feature organization.
- Compatibility with supervised and reinforcement learning workflows.
- Simplified dataset version management.
- Improved reproducibility of experimental results.

The schema therefore represents considerably more than a simple column definition. It serves as the foundational layer upon which every subsequent component of the RSNSD generation framework is constructed.

5.7 Feature Metadata Model

The proposed RSNSD framework adopts a metadata-driven approach in which every dataset attribute is represented as an independent metadata object rather than a simple column definition. This design philosophy enables the dataset generation framework to treat features as self-describing entities that encapsulate not only the generated values but also their semantic meaning, structural properties, and machine learning significance.

Each dataset attribute is represented using a dedicated `Feature` object that serves as the fundamental building block of the schema. Rather than storing only the feature name, the metadata model maintains comprehensive information describing the behaviour and

characteristics of every dataset attribute. Consequently, every module within the framework—including traffic generation, feature extraction, schema validation, documentation generation, and machine learning—operates using the same unified metadata specification.

Figure 4 illustrates the relationship between the Feature object and its associated metadata components.

Figure 4: Metadata model adopted for feature representation in RSNSD.

The metadata associated with every feature consists of multiple independent components, each responsible for describing a specific aspect of the feature. Collectively, these metadata elements completely define the semantic and operational characteristics of every dataset column.

5.7.1 Feature Definition

The `Feature` object represents an individual dataset attribute and acts as the primary metadata container throughout the framework. Every generated column corresponds to exactly one Feature definition.

Each Feature object maintains the following information:

- Unique feature identifier.
- Human-readable feature description.
- Logical feature category.
- Data type.
- Measurement unit.
- Numerical bounds.
- Nullability.
- Normalization strategy.
- Source of feature generation.
- Machine learning role.
- Derived feature indicator.

By encapsulating all metadata within a single object, the framework eliminates ambiguity regarding feature interpretation while significantly simplifying future dataset extensions.

5.7.2 Feature Categories

Each feature belongs to a predefined feature category represented by the `FeatureGroup` enumeration. Rather than organizing features according to implementation order, RSNSD classifies attributes according to their semantic responsibility within the networking stack.

The defined feature groups include:

- Layer-2 Features
- Layer-3 Features
- Layer-4 Features
- Flow Features
- Temporal Features
- Statistical Features
- Quality of Service Features
- Context Features
- Label Features

This classification enables modular organization of the schema while allowing individual feature groups to evolve independently.

5.7.3 Data Type Specification

Every feature explicitly specifies its underlying data type using the `DataType` enumeration. Unlike loosely typed datasets where data interpretation depends on external documentation, RSNSD enforces explicit typing for every generated attribute.

The supported data types include:

- Integer
- Floating Point
- Boolean
- String
- Categorical
- Date-Time

Explicit datatype specification simplifies validation, serialization, feature extraction, and machine learning preprocessing.

5.7.4 Normalization Strategy

Machine learning algorithms often require numerical features to undergo preprocessing prior to training. Instead of embedding normalization logic within downstream learning pipelines, RSNSD records the preferred normalization strategy as part of the feature metadata.

The framework currently supports the following normalization methods:

- No normalization
- Standard Score Normalization
- Min-Max Scaling
- Logarithmic Scaling

Storing normalization information alongside feature definitions ensures that preprocessing pipelines remain reproducible and consistent across multiple experiments.

5.7.5 Feature Source

Every generated feature records the software component responsible for producing its value through the `FeatureSource` metadata.

Typical feature sources include:

- Packet Parser
- Flow Extractor
- Context Generator
- QoS Generator
- Label Generator
- Dataset Generator

Explicitly recording feature provenance improves traceability by allowing researchers to identify the origin of every dataset attribute.

5.7.6 Machine Learning Role

Not every dataset attribute serves the same purpose during model training. Consequently, each feature specifies its intended machine learning role using the `MLRole` enumeration.

The framework distinguishes between:

- Predictive input features.
- Supervisory labels.
- Metadata attributes.

This separation prevents accidental inclusion of target variables during model training while simultaneously supporting multiple learning paradigms including supervised classification and reinforcement learning.

5.7.7 Advantages of Metadata-driven Representation

The metadata-centric design adopted by RSNSD provides several practical and research-oriented advantages over conventional flat dataset representations.

1. Every feature becomes self-describing.
2. Dataset validation can be fully automated.
3. Documentation is generated directly from the schema.
4. Machine learning preprocessing becomes reproducible.
5. Future feature additions require minimal implementation effort.
6. Multiple dataset versions remain structurally compatible.
7. Schema evolution is independent of traffic generation logic.

Consequently, the metadata model forms the foundation upon which the remaining components of the dataset generation framework are constructed.

5.8 Feature Classification and Organization

The complete RSNSD feature space has been organized into multiple semantically independent feature groups rather than maintaining a single flat collection of dataset attributes. This hierarchical organization improves dataset readability, simplifies feature engineering, facilitates schema maintenance, and enables researchers to selectively utilize subsets of features depending on the target machine learning task.

Instead of classifying features solely according to protocol layers, the proposed organization considers both networking semantics and machine learning relevance. Consequently, features describing packet headers, flow behaviour, Quality of Service, contextual information, and supervisory labels are maintained independently while remaining part of a unified schema.

The complete feature space has been partitioned into the following feature groups:

1. Layer-2 Features
2. Layer-3 Features
3. Layer-4 Features
4. Flow Features
5. Temporal Features
6. Statistical Features
7. Quality of Service Features
8. Context Features
9. Label Features

Figure 5 illustrates the hierarchical organization adopted by the proposed dataset.

Figure 5: Hierarchical classification of RSNSD feature groups.

Unlike conventional traffic datasets that emphasize protocol headers, RSNSD deliberately combines protocol-level features with service-aware contextual information and network state descriptors. This enables machine learning models to learn both intrinsic traffic behaviour and the environmental conditions influencing network performance.

The following subsections discuss each feature group in detail.

5.8.1 Layer-2 Features

Layer-2 features describe the Ethernet characteristics of generated network traffic. Although network slicing decisions are generally performed above the data link layer, several Layer-2 attributes remain valuable for Software Defined Networking (SDN), OpenFlow forwarding, VLAN prioritization, and future network virtualization research.

The objective of including Layer-2 features is not to maximize the number of Ethernet header fields but rather to preserve only those attributes capable of influencing forwarding behaviour, Quality of Service policies, or traffic isolation.

The Layer-2 feature set therefore remains intentionally compact while preserving compatibility with future Software Defined Networking deployments.

Table 1 summarizes the Layer-2 features incorporated within RSNSD.

Table 1: Layer-2 Features of RSNSD

Feature	**Description**	**Data Type**	**Unit**	**Source**	**Normalizati**
`ether_type`	Ethernet frame type	Category	–	Packet Parser	None
vlan_present	VLAN tag present	Boolean	–	Packet Parser	None
vlan_priority	IEEE 802.1p VLAN priority	Integer	Priority	Packet Parser	None
broadcast_frame	Broadcast frame indicator	Boolean	–	Packet Parser	None
multicast_frame	Multicast frame indicator	Boolean	–	Packet Parser	None

Table 2: Layer-3 Features of RSNSD

Feature	Description	Data Type	Unit	Source	Normalizatio
ip_version	IPv4/IPv6 version	Category	–	Packet Parser	None
ttl	Time-to-live / Hop limit	Integer	Hops	Packet Parser	None

Feature	Description	Data Type	Unit	Source	Normalizatio
dscp	Differentiated Services Code Point	Integer	–	Packet Parser	None
ecn	Explicit Congestion Notification	Integer	–	Packet Parser	None
ip_packet_length	Total IP packet length	Integer	Bytes	Packet Parser	Log
fragmented	Fragmentation indicator	Boolean	–	Packet Parser	None
ip_header_length	IP header length	Integer	Bytes	Packet Parser	None
hop_limit	IPv6 hop limit	Integer	Hops	Packet Parser	None

Table 3: Layer-4 Features of RSNSD

Feature	Description	Data Type	Unit	Source	Normalizatio
source_port	Source transport port	Integer	–	Packet Parser	None
destination_port	Destination transport port	Integer	–	Packet Parser	None
transport_protocol	TCP/UDP protocol	Category	–	Packet Parser	None
tcp_flags	TCP flag combination	Category	–	Packet Parser	None
window_size	TCP window size	Integer	Bytes	Packet Parser	Log
retransmission_count	Number of retransmissions	Integer	Packets	Packet Parser	Log
segment_size	TCP segment size	Integer	Bytes	Packet Parser	Log
acknowledgement_ratio	ACK ratio	Float	Ratio	Packet Parser	Standard
connection_state	Connection status	Category	–	Packet Parser	None
transport_latency	Estimated transport latency	Float	ms	Packet Parser	Standard

Table 4: Flow Features of RSNSD

Feature	Description	Data Type	Unit	Source	Normalizatio
flow_id	Unique flow identifier	String	–	Flow Extractor	None
flow_duration_ms	Flow duration	Float	ms	Flow Extractor	Standard

Feature	Description	Data Type	Unit	Source	Normalizatio
packet_count	Number of packets	Integer	Packets	Flow Extractor	Log
byte_count	Number of bytes	Integer	Bytes	Flow Extractor	Log
forward_packet_count	Forward packets	Integer	Packets	Flow Extractor	Log
backward_packet_count	Reverse packets	Integer	Packets	Flow Extractor	Log
forward_byte_count	Forward bytes	Integer	Bytes	Flow Extractor	Log
backward_byte_count	Reverse bytes	Integer	Bytes	Flow Extractor	Log
packet_rate_pps	Packet rate	Float	Packets/s	Flow Extractor	Standard
byte_rate_bps	Byte rate	Float	Bytes/s	Flow Extractor	Standard
flow_start_time	Flow start timestamp	DateTime	UTC	Flow Extractor	None
flow_end_time	Flow end timestamp	DateTime	UTC	Flow Extractor	None

Table 5: Temporal Features of RSNSD

Feature	Description	Data Type	Unit	Source	Normalizatio
flow_start_hour	Hour of flow start	Integer	Hour	Flow Extractor	None
flow_day_of_week	Day of week	Category	–	Flow Extractor	None
flow_month	Month	Integer	Month	Flow Extractor	None
inter_arrival_mean	Mean inter-arrival time	Float	ms	Flow Extractor	Standard
inter_arrival_std	Std. inter-arrival time	Float	ms	Flow Extractor	Standard
active_duration	Active flow duration	Float	s	Flow Extractor	Standard
idle_duration	Idle duration	Float	s	Flow Extractor	Standard
burst_duration	Burst duration	Float	s	Flow Extractor	Standard

Table 6: Statistical Features of RSNSD

Feature	Description	Data Type	Unit	Source	Normalizatio
packet_size_mean	Mean packet size	Float	Bytes	Flow Extractor	Standard
packet_size_std	Std. packet size	Float	Bytes	Flow Extractor	Standard
packet_size_min	Minimum packet size	Integer	Bytes	Flow Extractor	None
packet_size_max	Maximum packet size	Integer	Bytes	Flow Extractor	None
packet_size_variance	Packet size variance	Float	$Bytes^2$	Flow Extractor	Standard
flow_entropy	Flow entropy	Float	–	Flow Extractor	Standard
forward_backward_ratio	Fwd/Bwd packet ratio	Float	Ratio	Flow Extractor	Standard
burstiness_index	Burstiness index	Float	–	Flow Extractor	Standard
coefficient_variation	Coefficient of variation	Float	–	Flow Extractor	Standard
packet_size_median	Median packet size	Float	Bytes	Flow Extractor	Standard

Table 7: QoS Features of RSNSD

Feature	Description	Data Type	Unit	Source	Normalizatio
latency_ms	End-to-end latency	Float	ms	QoS Generator	Standard
jitter_ms	Packet delay variation	Float	ms	QoS Generator	Standard
packet_loss_pct	Packet loss	Float	%	QoS Generator	Standard
throughput_mbps	Throughput	Float	Mbps	QoS Generator	Standard
bandwidth_mbps	Allocated bandwidth	Float	Mbps	QoS Generator	Standard
availability_pct	Service availability	Float	%	QoS Generator	None
sla_compliance	SLA compliance status	Boolean	–	QoS Generator	None
delay_budget_ms	Allowed delay budget	Float	ms	QoS Generator	None
reliability_score	Reliability score	Float	–	QoS Generator	Standard

Feature	Description	Data Type	Unit	Source	Normalizatio
resource_utilization	Resource utilization	Float	%	QoS Generator	MinMax

Table 8: Context Features of RSNSD

Feature	**Description**	**Data Type**	**Unit**	**Source**	**Normalizati**
service_type	High-level service category	Category	–	Context Generator	None
application_name	Application generating the traffic	String	–	Context Generator	None
slice_type	Target network slice type	Category	–	Context Generator	None
service_priority	Priority assigned to the service	Integer	Level	Context Generator	None
network_load_pct	Overall network utilization	Float	%	Context Generator	Min-Max
cell_load_pct	Serving cell utilization	Float	%	Context Generator	Min-Max
active_users	Number of active users in the serving cell	Integer	Users	Context Generator	Log
device_type	Type of user equipment	Category	–	Context Generator	None
mobility_state	Current mobility profile	Category	–	Context Generator	None
time_of_day	Time period of generated traffic	Category	–	Context Generator	None
signal_strength_dbm	Received signal strength	Float	dBm	Context Generator	Standard
sinr_db	Signal-to-Interference-plus-Noise Ratio	Float	dB	Context Generator	Standard
rsrp_dbm	Reference Signal Received Power	Float	dBm	Context Generator	Standard
rsrq_db	Reference Signal Received Quality	Float	dB	Context Generator	Standard
cqi	Channel Quality Indicator	Integer	Index	Context Generator	None
network_slice_id	Serving network slice identifier	Category	–	Context Generator	None
slice_resource_utilization	Current slice resource utilization	Float	%	Context Generator	Min-Max
radio_resource_blocks_used	Allocated Physical Resource Blocks	Integer	PRBs	Context Generator	Log
handover_in_progress	Whether UE is undergoing handover	Boolean	–	Context Generator	None

Table 9: Label Features of RSNSD

Feature	Description	Data Type	Unit	Source	Norm(n)	ML Role
service_class	Ground truth service category	Category	–	Label Generator	None	Label
application_id	Unique application identifier	Category	–	Label Generator	None	Label
slice_type_label	Ground truth target network slice	Category	–	Label Generator	None	Label
priority_level	Ground truth service priority	Integer	Level	Label Generator	None	Label
allocated_slice	Slice allocated by the scheduling policy	Category	–	Label Generator	None	Label
sla_satisfied	Whether QoS requirements were satisfied	Boolean	–	Label Generator	None	Label
reward_score	Reward assigned to the scheduling decision	Float	Score	Label Generator	None	Label